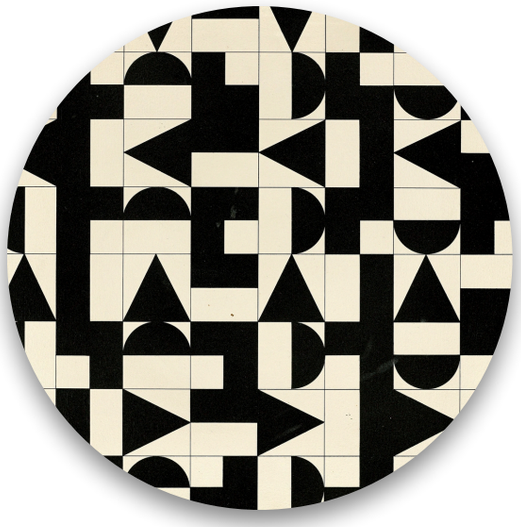
Generation Algorithms

Combinatorial Algorithms

Valentin Cassano

Guangdong Technion Israel Institute of Technology

Permutations



Jan Kubicek

Let us now turn our attention to a minimal change order for permutations.

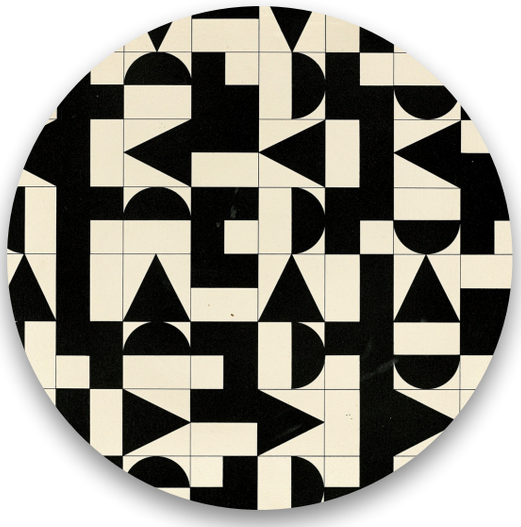
We begin by clarifying what this order looks like. Note that any two distinct permutations π and π' must differ in at least two positions. If π and π' differ in exactly two positions i and j, then π' can be obtained from π by a single transposition, that is, by exchanging the elements π_i and π_j in π. The change is minimal if the transposition is between two adjacent positions, that is, if $|i-j|=1$.

Equivalently, there is an $i \in [0:n-1)$ such that for all $j \in [0:n)$,

$$\pi'_j = \begin{cases} \pi_{i+1} & \text{if } j = i \\ \pi_i & \text{if } j = i+1 \\ \pi_j & \text{otherwise} \end{cases}$$

We say that a list of permutations is in minimal change order if every two consecutive permutations are related by this equation.

PERMUTATIONS



Jan Kubicek

The reason for looking at adjacent positions is explained as follows.

Counting the positions in which two permutations differ cannot tell a swap of neighbors from a swap of distant elements: both change exactly two positions. Thus, we can define a finer measure $d(\pi, \pi')$ as:

$d(\pi, \pi') =$ how many pairs of values appear in opposite order in π and π'

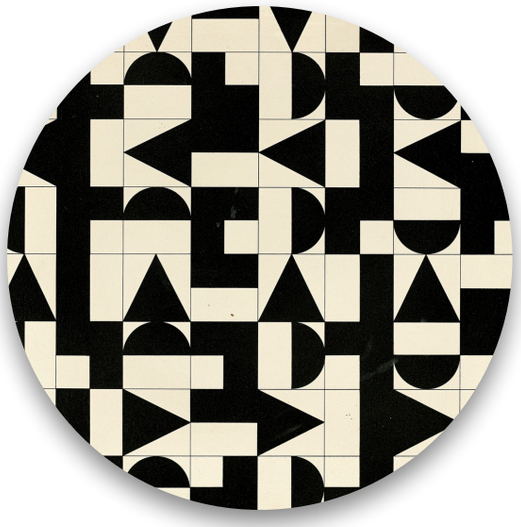
For example, let $\pi = [0,1,2]$ and swap two positions:

swapped positions	π	pairs in opposite order	d
0 and 1	[1, 0, 2]	(0,1)	1
1 and 2	[0, 2, 1]	(1,2)	1
0 and 2	[2, 1, 0]	(0,1) (0,2) (1,2)	3

In general, swapping positions $i < j$ with k elements between them gives $d = 2k + 1$: the swapped pair counts once, and each element in between forms one reversed pair with each of the two swapped elements.

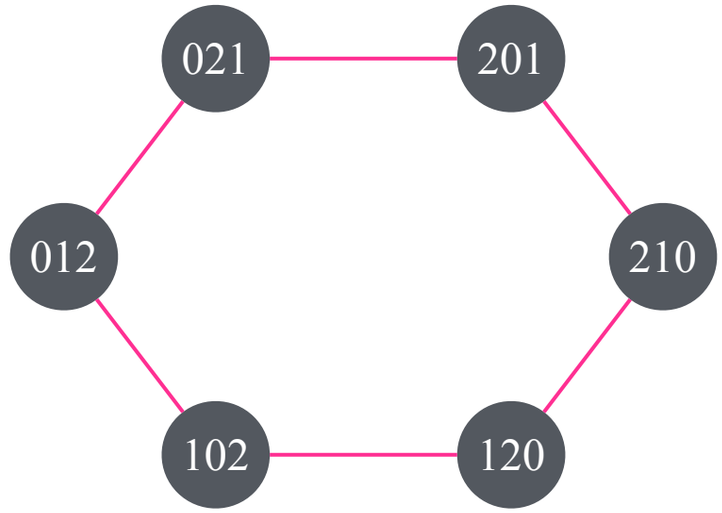
So $d = 1$ exactly when the swap is between adjacent positions, and 1 is the smallest nonzero distance. Minimal change means distance 1.

Permutations



Jan Kubicek

Consider the following graph



The vertices this graph are the permutations of the set $[0:3)$. The edges join two vertices when $d = 1$, that is, when one is an adjacent transposition of the other.

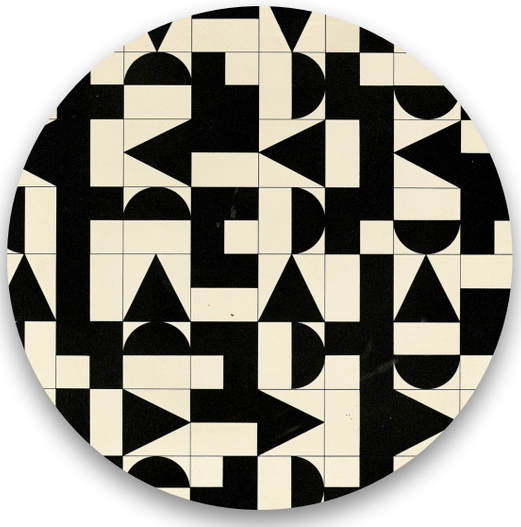
A minimal change order is a path through the graph that visits every vertex exactly once. The situation is similar to Gray codes being Hamiltonian paths in the hypercube.

The Trotter–Johnson list for this graph walks once around edges as:

$$012 \to 021 \to 201 \to 210 \to 120 \to 102$$

and the last permutation is adjacent to the first, so the order is a cycle.

Permutations



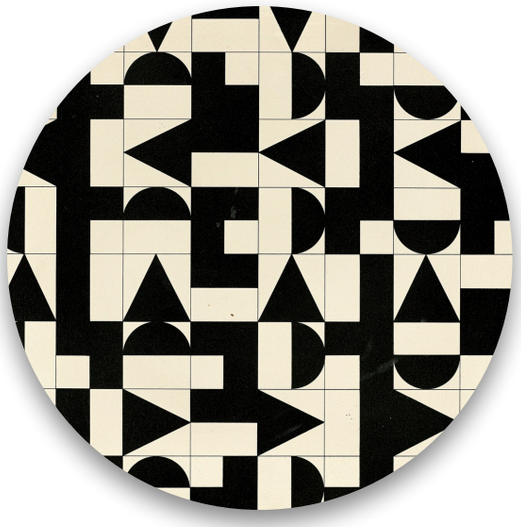
Jan Kubicek

The Trotter–Johnson algorithm is a nice example of an algorithm for generating the list $\mathbf{T}^n$ of the permutations in $\mathbf{\Pi}^n$ in the minimal change order. The algorithm is most easily described recursively.

The idea: the largest element sweeps back and forth.

Take the permutations of $\mathbf{T}^{n-1}$ one after the other. For each one, slide the new largest element $n-1$ across it, one position at a time, visiting every position. Then move on to the next permutation of $\mathbf{T}^{n-1}$ and sweep in the opposite direction.

PERMUTATIONS



Jan Kubicek

To understand the idea, take $\mathbf{T}^2 = [01,10]$ and consider $n-1 = 2$.
Then, we build $\mathbf{T}^3$ as follows:

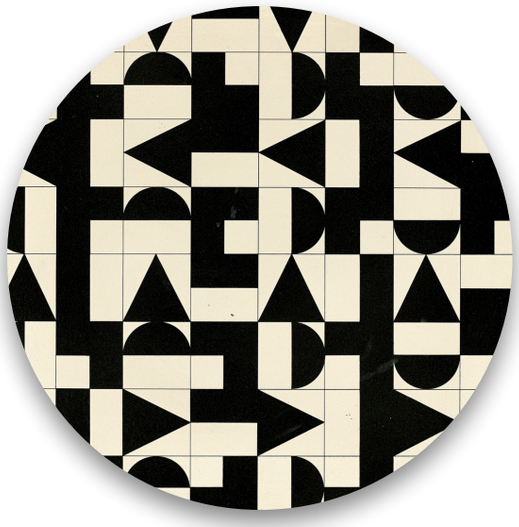
b	$T^2[b]$	sweep: 2	permutations
0	01	←	012 021 201
1	10	→	210 120 102

The sweep in alternate directions obeys the following principle.

While the largest element sweeps, nothing else moves. When the sweep ends, the largest element sits at one end; the others then take one step from $\mathbf{T}^2_b$ to $\mathbf{T}^2_{b+1}$, which is allowed because $\mathbf{T}^{n-1}$ is itself a minimal change order.

The next sweep must start where the last one stopped, so it must go back.

PERMUTATIONS



Jan Kubicek

The Trotter-Johnson algorithm is a nice example of an algorithm for generating the list $\mathbf{T}^n$ of the permutations in Π^n in the minimal change order is most easily described recursively as follows.

For the base case of the recursion, set $\mathbf{T}^1 = [[0]]$.

For the inductive case, build $\mathbf{T}^n$ from $\mathbf{T}^{n-1} = [\mathbf{T}_0^{n-1}, \ldots, \mathbf{T}_{(n-1)!-1}^{n-1}]$ by:

$|\mathbf{T}^{n-1}| = (n-1)!$

1. set $\mathbf{T}^n = []$
2. for each $\mathbf{T}_b^{n-1} \in \mathbf{T}^{n-1}$,
   (a) build a list $\mathbf{T}$ with n copies of $\mathbf{T}^{n-1}$ $|\mathbf{T}| = n$
   (b) for each permutation $\pi_t \in \mathbf{T}$,
      (i) build a permutation

$$\rho = \pi[0:p_{bt}) + [n-1] + \pi[p_{bt}:]$$

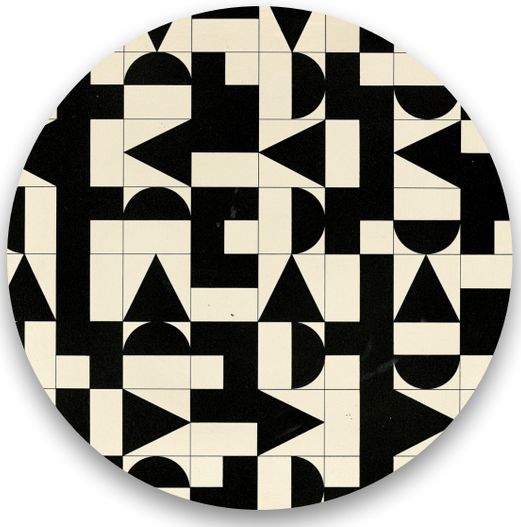
      where

$$p_{bt} = \begin{cases} n-(t+1) & \text{if } b \text{ iseven} \\ t & \text{if } b \text{ isodd} \end{cases}$$

      (ii) add ρ to $\mathbf{T}^n$

Note the rank of every permutation $\rho \in \mathbf{T}^n$ depends on the rank of the permutation $\pi \in \mathbf{T}^{n-1}$ from which it is build, plus the offset t indicating which copy of π in $\mathbf{T}$ we are currently looking at.

PERMUTATIONS



Jan Kubicek

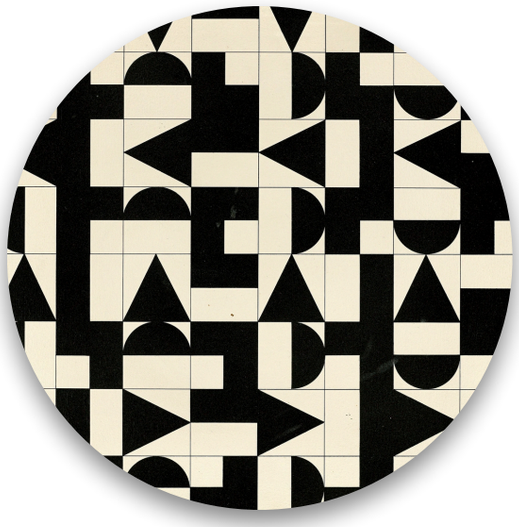
Let us compute all $\mathbf{T}^4$ by sweeping 3 through each permutation in $\mathbf{T}^3 = [012, 021, 201, 210, 120, 102]$. Then:

b	$\mathrm{T}^3[b]$	sweep	$t=0$	$t=1$	$t=2$	$t=3$
0	012	←	0123	0132	0312	3012
1	021	→	3021	0312	0231	0213
2	201	←	2013	2031	2301	3201
3	210	→	3210	2310	2130	2013
4	120	←	1203	1230	1320	3120
5	102	→	3102	1302	1032	1023

The block b and offset t give us a hint on how to compute the rank of a permutation. Precisely, the rank of an entry is $nb+t$.

In the example above: $\mathbf{T}_9^4 = [2,0,3,1]$ is in block $b = 2$, offset $t = 1$.

PERMUTATIONS



Jan Kubicek

Why does $\mathbf{T}^n$ contain every permutation exactly once?

A permutation π of $[0:n)$ is described exactly by two pieces of data:

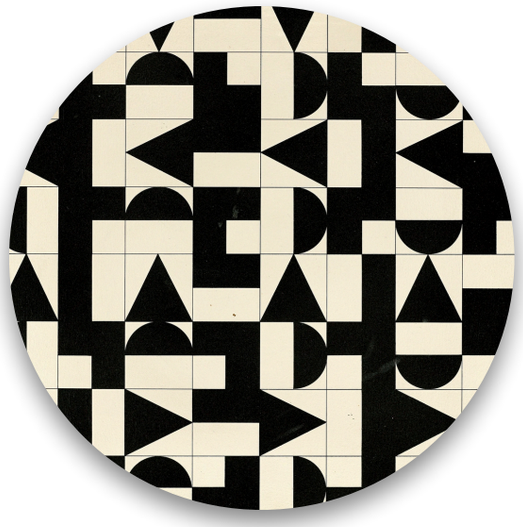
1. a permutation π' of $[0:n-1)$ obtained by removing $n-1$ from π;
2. the position k of $n-1$ in π, out of n possibilities.

Conversely, let π' be a permutation of $[0:n-1)$. Then, every pair (π', k) with $k \in [0:n)$ gives a permutation of$[0:n)$, by inserting $n-1$ into π' at position k.

So Π^n is in one-to-one correspondence with $\Pi^{n-1} \times [0:n)$, a set with $(n-1)! \cdot n = n!$ elements.

This is part of the correctness argument (nothing is missing and nothing is repeated). The point is that ***insert the largest element*** is a bijection between Π^n and $\Pi^{n-1} \times [0:n)$.

Permutations



Jan Kubicek

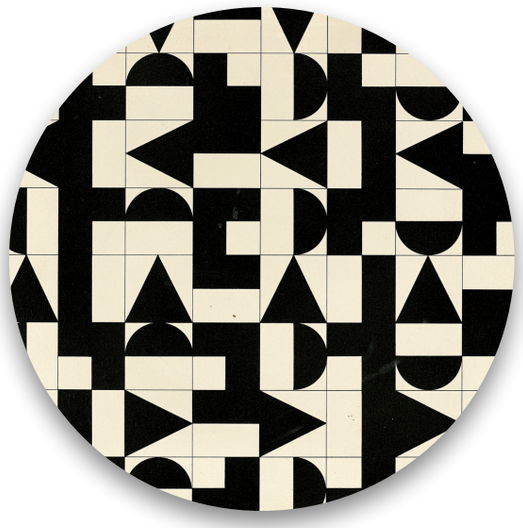
Why do consecutive permutations differ by an adjacent transposition?

The proof is by induction on n. Assume $\mathbf{T}^{n-1}$ is a minimal change order.

Inside a block. π does not change and $n-1$ moves one position, exchanging places with a neighbor.

Between two blocks. The parities alternate, so the sweeps are arranged head to tail. So $n-1$ does not move, and the rest changes from one permutation of $\mathbf{T}^{n-1}$ to the next, which are consecutive in $\mathbf{T}^{n-1}$ and therefore differ by an adjacent transposition (positions stay adjacent, because $n-1$ sits at an end).

Permutations



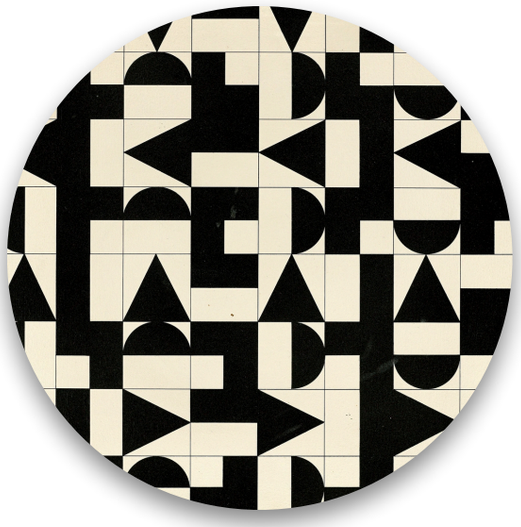
Jan Kubicek

```
def trotterjohnson(n:int) -> list[list[int]]:
    """
    the Trotter-Johnson enumeration of \Pi^n
    """
    # require: n >= 0


    # to be completed in class


    return T
    # ensure: T = T^n: T = T^n
```

Permutations



Jan Kubicek

Let us now turn our attention to a ranking algorithm for $\mathbf{T}^n$. The construction of this algorithm is based on the following observation.

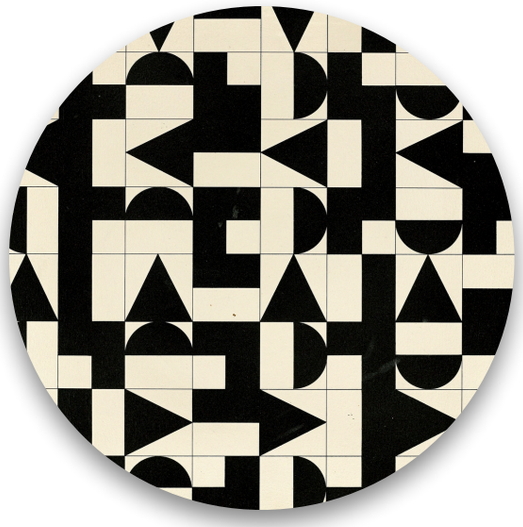
Let $\pi \in \mathbf{T}^n$ and $\pi_k = n-1$. Moreover, let $\pi' = \pi[:k] + \!\!+ \, \pi(k:]$. That is, π' is π with $n-1$ removed.

Then π is obtained from π' by inserting $n-1$ at position k.

By the construction, π lies in the block of π'. This block is numbered $b = \mathrm{rank}(\pi')$. Its offset inside that block is determined by k and by the parity of b.

It follows that we can obtain $\mathrm{rank}(\pi)$ recursively from the value of $\mathrm{rank}(\pi')$ together with that of k.

PERMUTATIONS



Jan Kubicek

Precisely, the ranking algorithm is defined as:

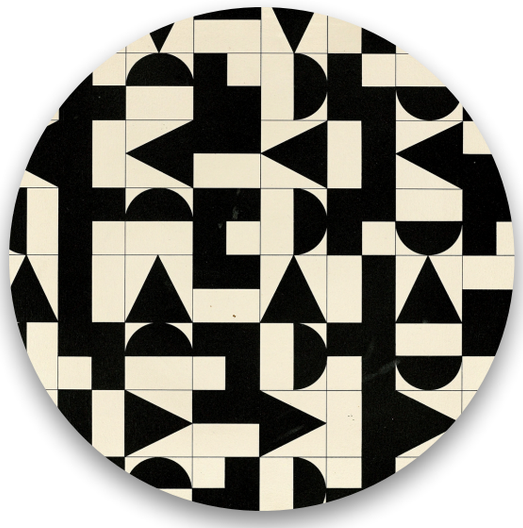
$$\begin{aligned}
\text{rank}([0],1) &= 0 \\
\text{rank}(\pi, n) &= n \cdot b + t \\
&\text{where} \quad b = \text{rank}(\pi', n-1) \\
&\qquad\quad k = \text{the position of } n-1 \text{ in } \pi \\
&\qquad\quad t = \begin{cases} n-(k+1) & \text{if } b \text{ is even} \\ k & \text{if } b \text{ is odd} \end{cases}
\end{aligned}$$

Reading the formula:

- $n \cdot b + t$ is a division with remainder: b says which block, and t says where inside the block.
- every block b has a sweep of n: right to left when n is even, and left to right when b is odd.

This is exactly the construction of the Trotter-Johnson algorithm.

PERMUTATIONS



Jan Kubicek

To compute $\mathbf{rank}([2,0,3,1],4)$, we strip the largest element repeatedly, then evaluate from the bottom up.

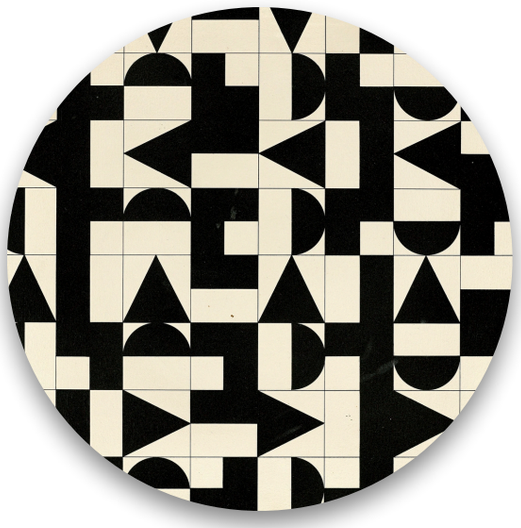
n	π	k	π'	b	t	$nb + t$
1	0					0 (base case)
2	01	1	0	0 (even)	2-1-1 = 0	0
3	201	0	01	0 (even)	3-1-0 = 2	2
4	2031	2	201	2 (even)	4-1-2 = 1	9

rank(π')

rank(π)

This means $\mathbf{rank}([2,0,3,1],4) = 9$, and indeed $\mathbf{T}_9^4 = [2,0,3,1]$.

Permutations



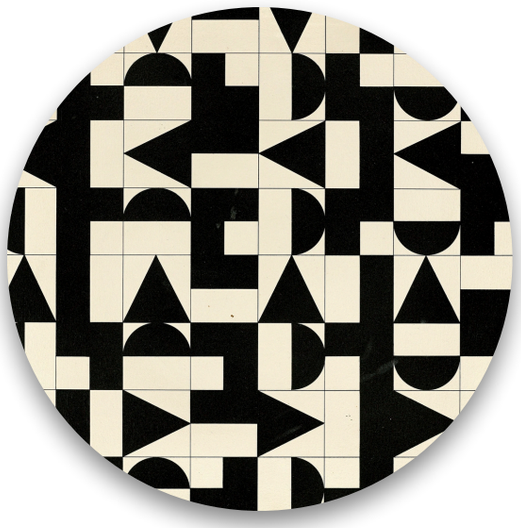
Jan Kubicek

```
def rank(pi:list[int], n:int) -> int:
    """
    the position of pi in T^n
    """
    # require: pi in \Pi^n


    # to be completed in class


    return r
    # ensure: T^n[r] = pi
```

PERMUTATIONS



Jan Kubicek

Unranking is defined recursively following an inverse procedure.

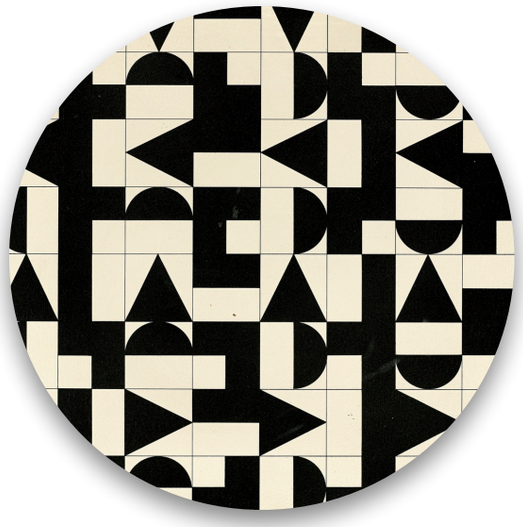
For the base case, that is $r = 0$ and $n = 1$, set $\mathbf{unrank}(0,1) = [0]$.

For the inductive case, that is $n \geq 2$:

1. set $b = r \div n$ and $t = r \mod n$ (division with remainder);
2. set $\pi' = \mathbf{unrank}(b, n-1)$;
3. set $k = n - (t+1)$ if b is even, and $k = t$ if b is odd;
4. set $pi = \pi'[0:k) + [n-1] + \pi[k:]$

Intuitively, b tells us which permutation π' of $\mathbf{T}^{n-1}$ we are in, t tells us how far the largest element has swept, and the parity of b tells us in which direction it was sweeping.

Permutations



Jan Kubicek

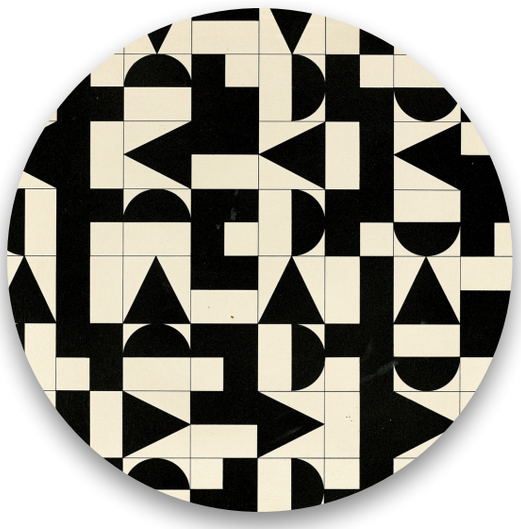
Let us compute $\mathbf{unrank}(9,4)$, we strip the largest element repeatedly, then evaluate from the bottom up.

		$r \div n$	$r \bmod n$	$\mathrm{unrank}(b, n-1)$		
n	r	b	t	π'	k	π
4	9	2	1	201	2	2301
3	2	0	2	01	0	201
2	0	0	0	0	1	01
1	0					0

The rows are computed top to bottom (to find b, t and which smaller permutation is needed), and the permutations are rebuilt bottom to top.

This means $\mathbf{unrank}(9,4) = [2,0,3,1]$, and indeed $\mathbf{T}_9^4 = [2,0,3,1]$.

Permutations



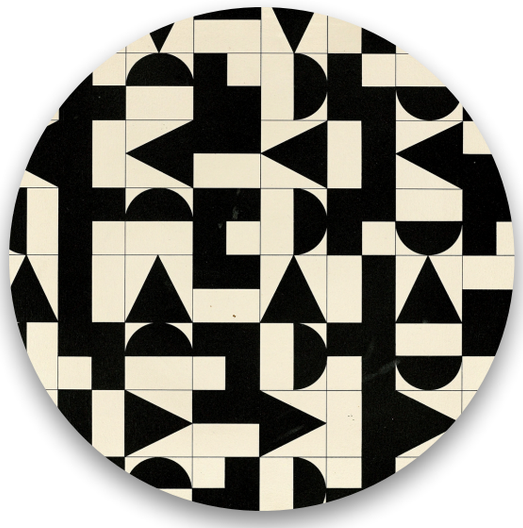
Jan Kubicek

```
def unrank(r:int, n:int) -> list[int]:
    """
    the rth permutation in T^n
    """
    # require: r in [0:n!)


    # to be completed in class


    return pi
    # ensure: T^n[r] = pi
```

Permutations



Jan Kubicek

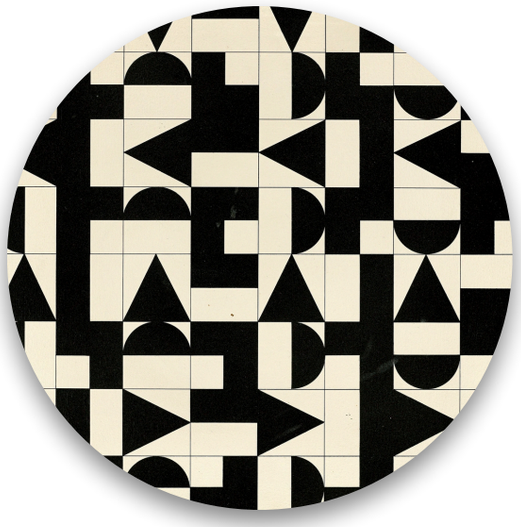
The successor algorithm, that is, computing $\rho = \mathbf{T}^n_{r+1}$ from $\pi = \mathbf{T}^n_r$, is somewhat more complicated.

To carry out this computation, let $\pi_k = n-1$ and $\pi' = \pi[0:k) + \pi(k:]$. There are four cases to consider:

	rank(π')	k	succesor
1	even	> 0	swap the values in positions k and k-1
2	even	$= 0$	add n-1 as the first element of successor(π', n-1)
3	odd	$< n$-1	swap the values in positions k and k+1
4	odd	$= n$-1	add n-1 as the last element of successor(π', n-1)

For $n = 2$ the successor just swaps the two elements; the recursion thus ends there, and the last permutation wraps around to the first.

PERMUTATIONS



Jan Kubicek

The four cases to consider are justified as follows:

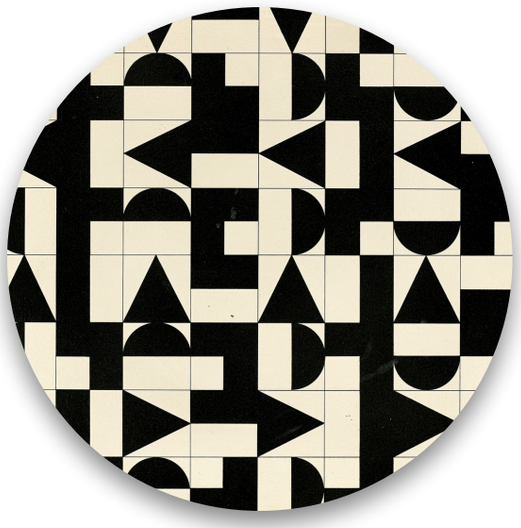
Let $r = \mathrm{rank}(\pi) = bn + t$, with $b = \mathrm{rank}(\pi')$. The successor has rank $r + 1 = bn + (t + 1)$. Then, two things can happen.

(i) $t + 1 < n$: the successor of π is in the same block. Same π', offset $t + 1$, so $n-1$ takes one more step of its sweep: to the left if b is even, to the right if b is odd. These are cases 1 and 3.

(ii) $t + 1 = n$: then, π is the last permutation of its block. We have $n-1$ is at the end of its sweep, and $r + 1 = (b + 1)n + 0$ is the first permutation of the next block. That block has the opposite parity, so offset 0 puts $n-1$ at the very same end, and the rest is $\mathbf{T}_{b+1}^{n-1} = \mathrm{successor}(\pi', n-1)$. These are cases 2 and 4.

From the offset t to the position k. The proof above splits on t, but the rules on the previous slide are stated with the position k of $n-1$. This is because k can be read off π while t cannot. The two, however, are linked: $t = n - (k + 1)$ if b is even; and $t = k$ if b is odd. So each condition on t becomes a condition on k.

Permutations



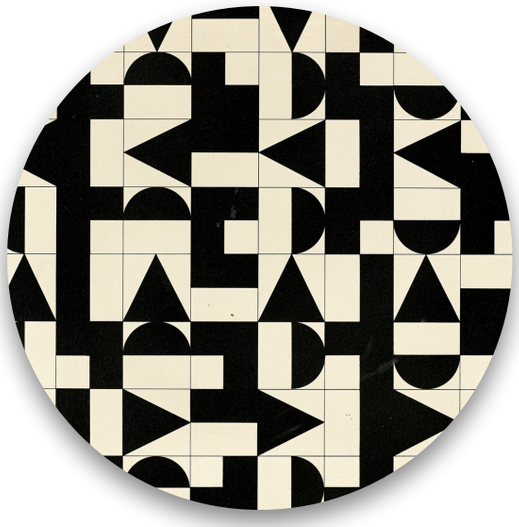
Jan Kubicek

Testing the parity of $\mathrm{rank}(\pi')$ without computing the rank.

The successor only needs to know whether $\mathrm{rank}(\pi')$ is even or odd. This is the parity of the number of inversions of π' (pairs $i < j$ such that $\pi'_i > \pi'_j$.

To see why, note the following. $\mathbf{T}_0^{n-1}$ is the identity, which has no inversions. Thus, its parity is 0. Consecutive permutations differ by an adjacent transposition, which creates or destroys exactly one inversion (the swapped pair). So the parity of the number of inversions flips at every step, exactly like the parity of the rank.

Permutations



Jan Kubicek

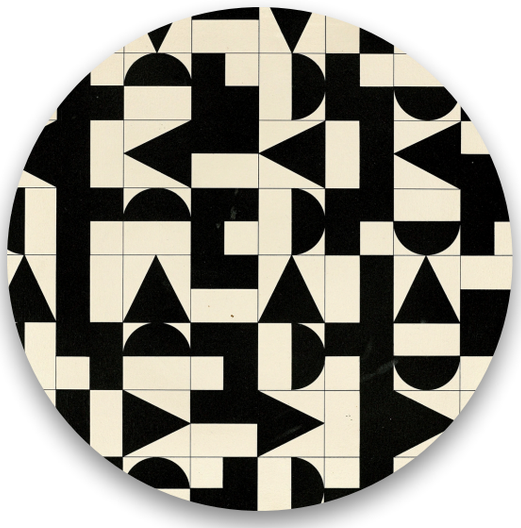
To compute the parity of the rank of a permutation, we do as follows.

By the previous slide, the parity of $\mathrm{rank}(\pi')$ is the parity of the number of inversions ofπ', so the rank itself is never needed. Let m be the length of π'. There are two ways:

1. **Count the inversions**. Compare every pair $i < j$. This takes $O(m^2)$.
2. Count the cycles. Start at an element not seen yet, follow $i \mapsto \pi_i$ until you return to the start, and count one cycle. Since every element is visited once, this takes $O(m)$.

Why cycles? A cycle of length ℓ can be sorted with $\ell - 1$ transpositions. This means π' is a product of $m - c$ transpositions, where c is the number of cycles. Each transposition flips the parity of the number of inversions, and the identity has 0 inversions. Thus, $\mathrm{inv}(\pi') \equiv m - c \pmod 2$.

Permutations

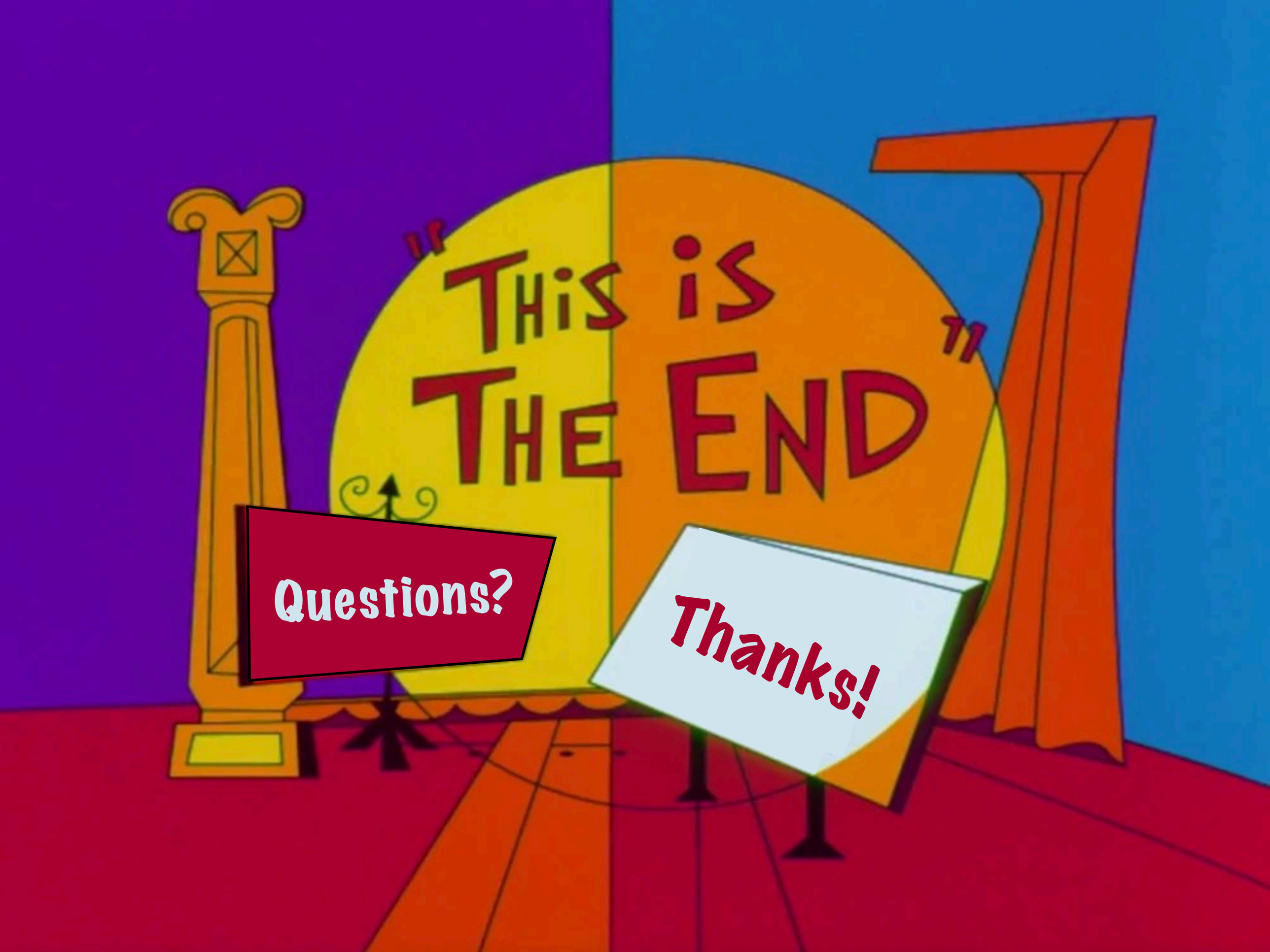


Jan Kubicek

```
def successor(pi:list[int], n:int) -> list[int]:
    """
    the next permutation in T^n
    """
    # require: pi in \Pi^n


    # to be completed in class


    return rho
    # ensure: rank(rho) = (rank(pi) + 1) mod n!
```


"This is The End"
Questions?
Thanks!